

Exponential and logarithmic time

So far we've considered only complexity functions $O(1)$, $O(n)$, $O(n^2)$ or $O(n^3)$, in general $O(n^k)$ for some whole number k . This week we'll consider other functions that characterize common algorithms, in particular,

exponential time

(very bad), typically found in algorithms that perform an exhaustive search.

logarithmic time

(very good), typically achieved by algorithms that cut the size of the problem at each step.

An important example of the latter class is the binary search algorithm.

We'll also need to consider a more general definition of big-O notation to accommodate these functions, and others.

Exponential time

Many algorithms operate by considering all possibilities in a set, a strategy called exhaustive search. This is often very expensive.

One variable: 2 rows

p	$p \wedge \neg p$
0	0
1	0

Satisfiability checking

The satisfiability checking problem is to determine whether, given a formula, there is an assignment of truth values to the variables that makes it true. For example, consider the following:

- $p \wedge \neg p$
- $(p \vee q) \wedge \neg p$
- $\neg p \wedge ((\neg q \vee r) \rightarrow p)$
- and so on

Two variables: 4 rows

p	q	$(p \vee q) \wedge \neg p$
0	0	0
0	1	1
1	0	0

One method for determining whether or not a formula is satisfiable is using truth tables, following directly from the definition. For formula f , containing variables $x_1, \dots, x_n$,

- Generate all rows of the truth table for f .
- If any row gives true, then f is satisfiable, else f is not satisfiable.

The input to the algorithm is the formula f . An appropriate measure of size is the number of distinct variables in f .

The main step is the generation of a row of the truth table.

- If there is only one variable, we must test 2 rows.
- If there are two variables, we must test $2 \times 2 = 4$ rows.
- If there are three variables, we must test $2 \times 2 \times 2 = 8$ rows.
- If there are four variables, we must test $2 \times 2 \times 2 \times 2 = 16$ rows.

For n variables, $2 \times 2 \times 2 \dots \times 2 = 2^n$ rows.

Subset Sum algorithm

The straightforward algorithm for the subset sum problem (a special case of the knapsack problem) is similar: try each selection and choose the largest one that fits. The example at right is has weights 23, 39, 57 and capacity 100.

For 3 items, the number of selections to check is $2 \times 2 \times 2 = 2^3 = 8$.

In general, for n items the number of selections to check is $2 \times 2 \times 2 \dots \times 2 = 2^n$.

As we'll see in week 8, the best known algorithms for these problems aren't significantly faster than exhaustive search.

Powers of two

1	1	0
---	---	---

Three variables: 8 rows

p	q	r	$\neg p \wedge ((\neg q \vee r) \rightarrow p)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Selection	Sum	
23 + 39 + 57	0	
23 + 39 + 57	57	
23 + 39 + 57	39	
23 + 39 + 57	96	(best)
23 + 39 + 57	23	
23 + 39 + 57	80	
23 + 39 + 57	62	
23 + 39 + 57	119	(too big)

The cost functions of the satisfiability tester and the straightforward knapsack algorithm are powers of two, which increase very steeply:

$2^1 = 2$	$= 2$	$2^6 = 2 * 2 * 2 * 2 * 2 * 2$	$= 64$
$2^2 = 2 * 2$	$= 4$	$2^7 = 2 * 2 * 2 * 2 * 2 * 2 * 2$	$= 128$
$2^3 = 2 * 2 * 2$	$= 8$	$2^8 = 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$	$= 256$
$2^4 =$ $2 * 2 * 2 * 2$	$=$ 16	$2^9 =$ $2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$	$= 512$
$2^5 =$ $2 * 2 * 2 * 2 * 2$	$=$ 32	$2^{10} =$ $2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2$	$=$ 1024

Every programmer should know the powers of 2. The approximation $2^{10} \approx 1000$ is also very useful.

Recall: properties of powers

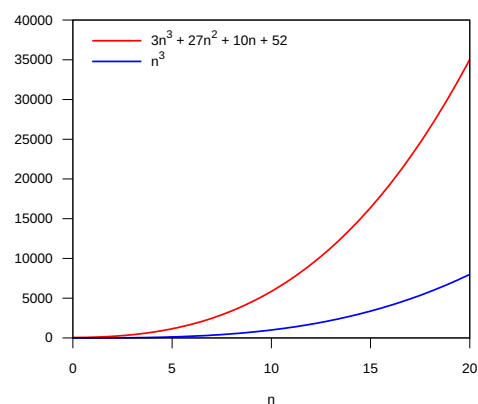
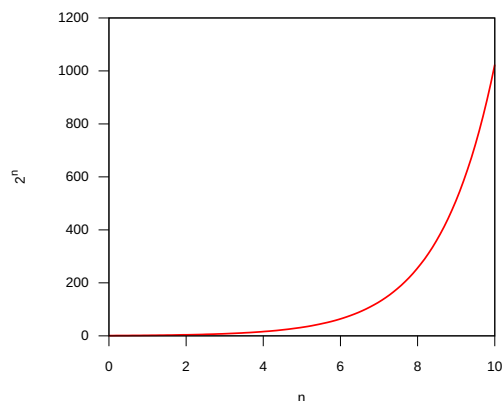
- $2^a * 2^b = 2^{a+b}$
- $(2^a)^b = 2^{a*b}$

Give approximate values for 2^{16} , 2^{30} and 2^{32} .

Big-O notation revisited

In Week 2 we discussed big-O notation for polynomial functions, yielding $O(n)$, $O(n^2)$, $O(n^3)$, ..., $O(n^k)$ for any positive whole number k . Now that we're dealing with other cost functions, such as 2^n , we need a generalized definition.

Consider the function $f(n) = 3n^3 + 27n^2 + 10n + 52$. When we say that this is $O(n^3)$, we mean that it grows no faster than n^3 . To compare the rates of growth of these two functions, we plot their ratio, $f(n)/n^3$.



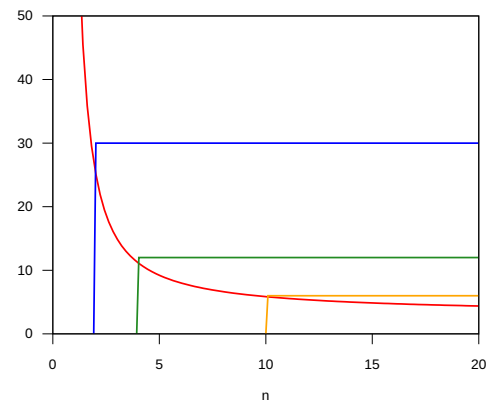
Two cubic functions

From the graph at right, we see that for large n , the ratio between the two functions has an upper bound:

For $n > 2$, $f(n)/n^3 \leq 30$ or equivalently $f(n) \leq 30 n^3$

For $n > 4$, $f(n)/n^3 \leq 12$ or equivalently $f(n) \leq 12 n^3$

For $n > 10$, $f(n)/n^3 \leq 6$ or equivalently $f(n) \leq 6 n^3$



Plot of $(3 n^3 + 27 n^2 + 10 n + 52)/n^3$

Many such bounds are possible; any of these will do.

Formal definition of big-O notation definition

A function $f(n)$ belongs to the complexity class $O(g(n))$ if there is a size N and a constant c such that

- for all $n > N$, $f(n) \leq c g(n)$



Big-O notation predates computer science by several decades. It was introduced by the German mathematician Paul Bachmann in 1894.

There are two constants here, both of which can be as big as we like:

N

which means only the behaviour for large n matters.

c

which means that we ignore constant factors.

A consequence of this definition is that if a function is $O(n^2)$, then it is also $O(n^3)$, $O(n^4)$, etc.

$$O(1) \subset O(n) \subset O(n^2) \subset O(n^3) \subset \dots \subset O(n^k) \subset \dots \subset O(2^n)$$

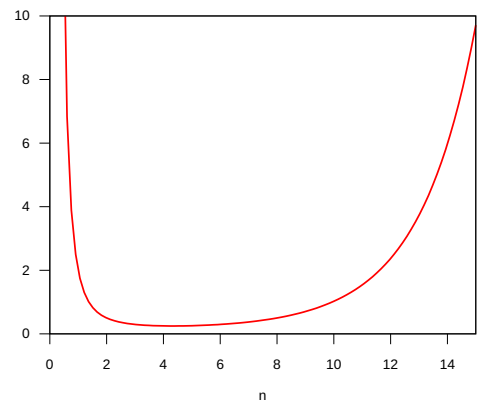
However we generally prefer the tightest class.

A function belongs to $O(g(n))$ if it grows no faster than $g(n)$. There is an associated notation, $\Theta(g(n))$, of functions that grow at the same rate as $g(n)$. (That's a capital Greek letter theta.)

Exponential growth

The graph at right compares the rate of growth of 2^n with that of n^3 , by plotting $2^n/n^3$.

- For small values of n , $2^n \leq n^3$.
- As n gets larger, the ratio between the two functions grows without limit.



Plot of $2^n/n^3$

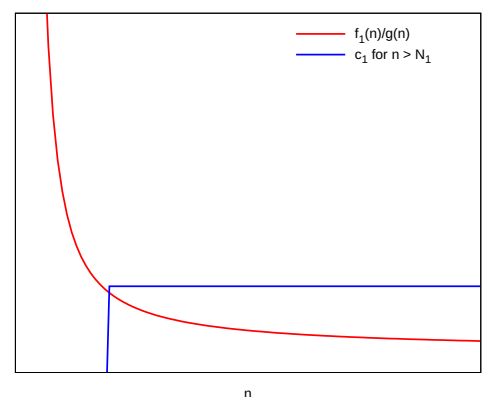
This means that the function 2^n is not $O(n^3)$. Indeed it is not $O(n^{10})$ or $O(n^{100})$ or $O(n^k)$ for any whole number k . It grows faster than any polynomial function.

This also means that any polynomial function (made up of constant powers of n), like $3n^3 + 27n^2 + 10n + 52$, is also $O(2^n)$. However, it is more informative to say it's $O(n^3)$.

Operations on order functions

Order notation simplifies calculations considerably. One common operation arises when an algorithm does one thing after another: we want to add the costs of the two parts.

For example:



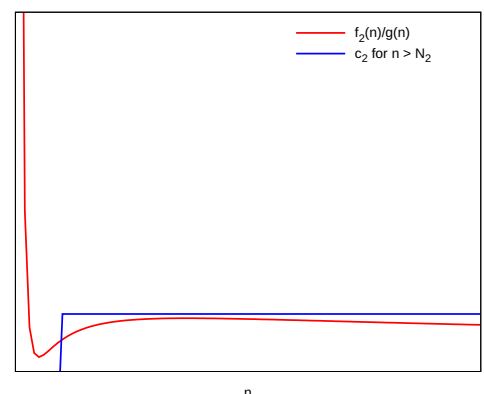
Property

If $f_1(n)$ and $f_2(n)$ are both $O(g(n))$, then $f_1(n) + f_2(n)$ is also $O(g(n))$.

Proof

From the definition, we know that there are constants N_1 , N_2 , c_1 and c_2 such that

- for all $n > N_1$, $f_1(n)/g(n) \leq c_1$, and
- for all $n > N_2$, $f_2(n)/g(n) \leq c_2$



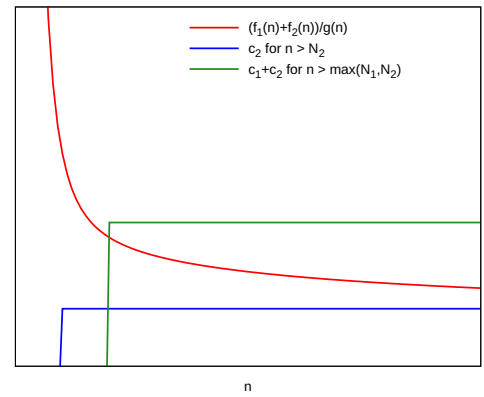
If N is the larger of N_1 and N_2 , then

- for all $n > N$, $(f_1(n) + f_2(n))/g(n) \leq c_1 + c_2$

i.e. $f_1(n) + f_2(n)$ is also $O(g(n))$.

For example,

- $4n^3 + 6n^3$ is $O(n^3)$, because both functions are $O(n^3)$.
- $3n^{20} + 2^n$ is $O(2^n)$, because both functions are $O(2^n)$. The faster-growing 2^n dominates n^{20}



We can use this to work out the complexity of methods from their parts. For example,

- If a method performs an operation taking $O(n^2)$ time followed by another operation taking $O(n^2)$ time, the whole thing takes $O(n^2)$ time.
- If a method performs an operation taking $O(n^3)$ time followed by another operation taking $O(2^n)$ time, the whole thing takes $O(2^n)$ time.

Searching an array

Last week we considered searching for a key in an array using the sequential search algorithm, which takes $O(n)$ time.

Recall sequential search:

n = 9	0	1	2	3	4	5	6	7	8
a	27	14	77	53	18	85	18	94	34
k	94								
i									

SequentialSearch(a[0..n])
 $i \leftarrow 0$
WHILE $i < n$ AND $a[i] \neq k$
 $i \leftarrow i+1$
RETURN i

Reset Single step Big step Run Stop

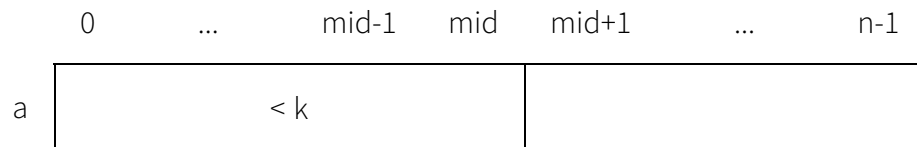
Binary search (logarithmic time)

If the array has been arranged in ascending order, we can do much better. Let **mid** be an index in the middle of the array:

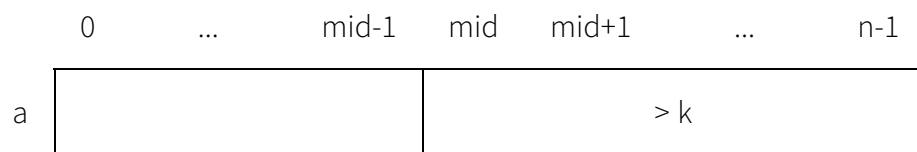


If we compare $a[mid]$ with k , there are three cases:

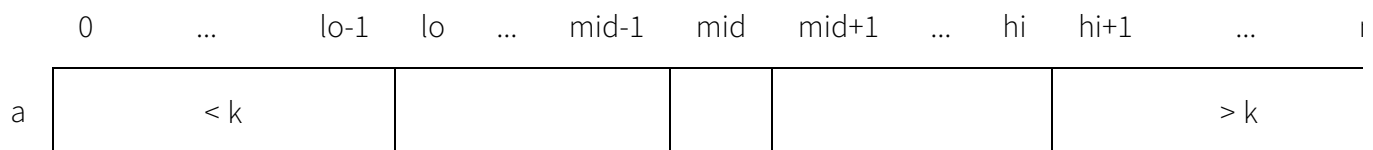
- If $a[mid] = k$, then we are done.
- If $a[mid] < k$, then we need only search $a[mid+1..n-1]$:



- If $a[mid] > k$, then we need only search $a[0..mid-1]$:

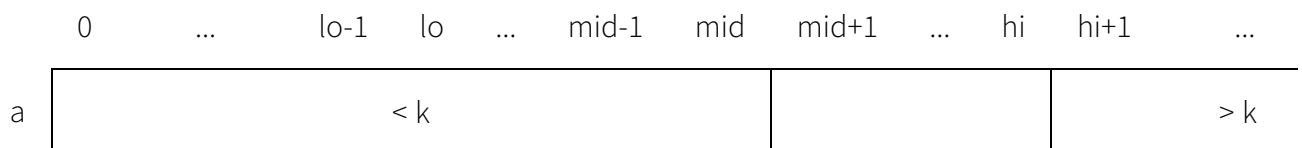


Repeating this process, we search smaller and smaller sub-arrays, until either we find k or end up with an empty search space. We need two variables **lo** and **hi** to keep track of the search area:

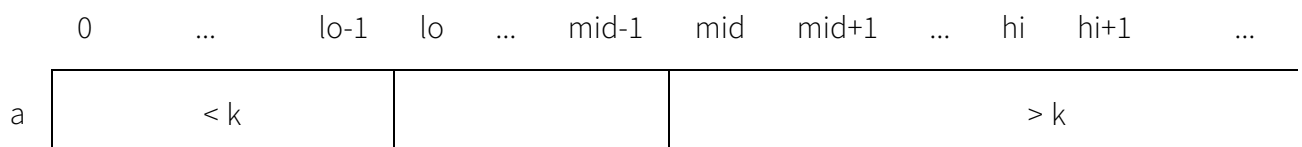


On each iteration, we compare k with $a[mid]$:

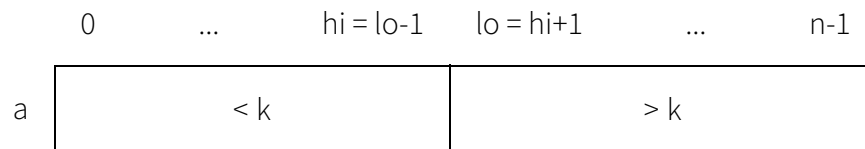
- If they are equal, we have found the entry we are looking for.
- If $a[mid] < k$, we can eliminate everything from mid down:



- If $a[mid] > k$, we can eliminate everything from mid up:



If the key we're looking for isn't in the array, **lo** and **hi** will eventually cross, and we'll have this situation:



Thus we have the following algorithm:

Algorithm BinarySearch(a[0..n-1], k)

lo ← 0

hi ← n-1

invariant: $0 \leq lo \leq hi+1 \leq n$ and $a[0..lo-1] < k < a[hi+1..n-1]$

WHILE lo ≤ hi

mid ← (lo+hi) DIV 2

IF a[mid] = k

RETURN mid

IF a[mid] < k

lo ← mid+1

ELSE

hi ← mid-1

RETURN -1

Try stepping through binary search:

n = 15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	14	16	22	27	34	36	40	47	51	58	62	63	77	85	93

k	62
lo	
hi	
mid	

Search key: 62

Example

Consider searching for 62 in this array:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	14	16	22	27	34	36	40	47	51	58	62	63	77	85	93

Stepping through the values of the variables **lo** and **hi** on each iteration:

	lo	hi	
Before entering the loop	0	14	mid = 7, examine a[7] = 47
After the first iteration	8	14	mid = 11, examine a[11] = 63
After the second iteration	8	10	mid = 9, examine a[9] = 58
After the third iteration	10	10	mid = 10, examine a[10] = 62, target found

So we found the key after examining 4 elements of the array.

Now consider searching for an element that isn't in the array, say 42:

	lo	hi	
Before entering the loop	0	14	mid = 7, examine a[7] = 47
After the first iteration	0	6	mid = 3, examine a[3] = 27
After the second iteration	4	6	mid = 5, examine a[5] = 36
After the third iteration	6	6	mid = 6, examine a[6] = 40
After the fourth iteration	7	6	exit the loop

Again we examined 4 elements of the array.

Analysis of binary search

The loop here has two control variables, **lo** and **hi**.

- The size of the input is the length of the list, n.
- The fundamental step is examination of an element of the array, which is the same as an iteration of the loop.
- On each iteration of the loop, we halve $hi-lo+2$ (initially

n+1) until it reaches 1:

	lo	hi	hi-lo+2
Before entering the loop	0	14	16
After the first iteration	0	6	8
After the second iteration	4	6	4
After the third iteration	6	6	2
After the fourth iteration	7	6	1

How many times do we have to halve the size?

Reminder: Logarithms

The logarithm of a number is the exponent to which another fixed value, the base, must be raised to produce that number.

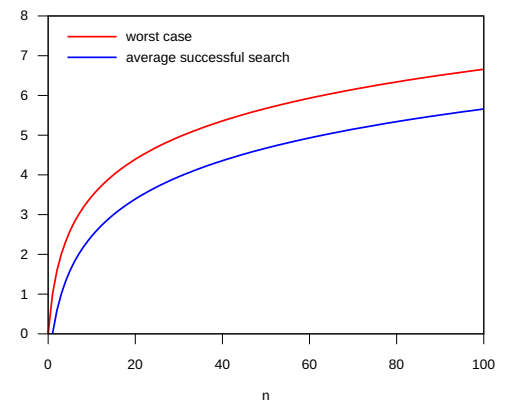
We shall assume the base is 2.

- For example, $\log(32)$, the logarithm of 32 to base 2 is 5, because $32 = 2^5$
- More generally, if $y = 2^x$, then $x = \log(y)$

That is, the logarithm graph is the mirror image of the exponential graph about the diagonal. Logarithmic complexity is as good and exponential complexity is bad. So $O(\log n)$ grows much more slowly than $O(n)$.

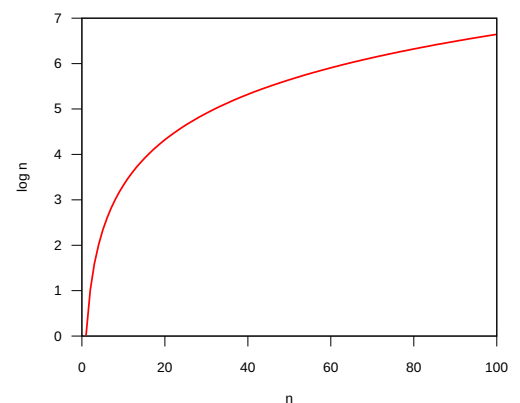
Actually this is $\log_2(y)$. It's possible to have logarithms to other bases, but it can be shown that they are a constant factor times the logarithm to base 2, and therefore also $O(\log n)$.

What are the approximate values of $\log(1000)$ and $\log(1,000,000)$?



Performance of binary search.

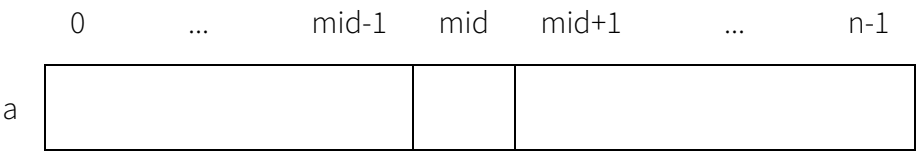
Surprising fact: successful searches take on average only one step less than the worst case. (Why?)



A variation on binary search

We have seen that although the equality test allows the binary search to terminate early, that yields little gain in

practice. So let's consider a variation omitting that test. Let **mid** be an index in the middle of the array:



Now when we test $a[\text{mid}] < k$, there are only two cases:

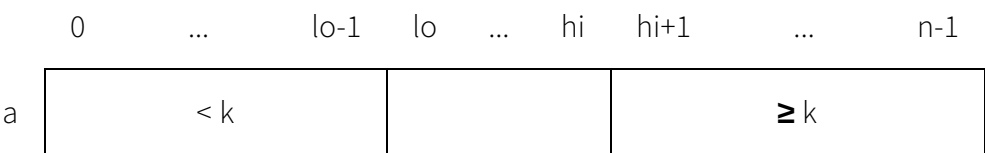
- If $a[\text{mid}] < k$, then we search $a[\text{mid}+1..n-1]$:



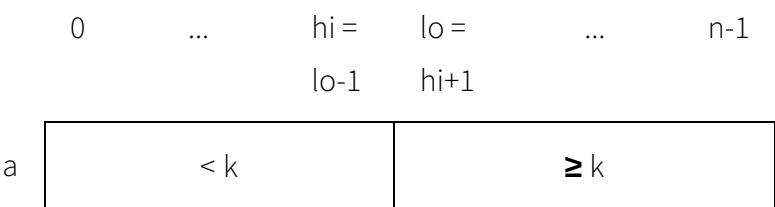
- If $a[\text{mid}] \geq k$, then we search $a[0..mid-1]$:



At we repeat the process, at each stage the array looks like this:



Eventually **lo** and **hi** will cross, and the loop will terminate with this situation:



Then if **k** is in the array, it will be at position **lo**. But compared to the original algorithm, the final value of **lo** contains additional information:

- If **k** is not in the array, then **lo** is the position where it would be inserted.
- If there are several copies of **k** in the array, then **lo** is

the position of the first one.

Here is the modified algorithm:

```
Algorithm AltBinarySearch(a[0..n-1], k)

  lo ← 0
  hi ← n-1

  invariant:  $0 \leq lo \leq hi+1 \leq n$  and  $a[0..lo-1] < k \leq a[hi+1..n-1]$ 

  WHILE lo ≤ hi
    mid ← (lo+hi) DIV 2
    IF a[mid] < k
      lo ← mid+1
    ELSE
      hi ← mid-1
  RETURN lo
```

Its worst case time is the same as the original algorithm, $O(\log n)$. Unlike the original, it always takes that much time. On the other hand, it only does one comparison each time round the loop, and it gives more information (as discussed above).

Try stepping through the modified binary search:

n = 15	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	14	16	22	27	34	36	40	47	51	58	62	63	77	85	93
k	62														
lo															
hi															
mid															

Search key:

To see the difference with the original algorithm, try searching for 47.

Complexity classes

Now we can do a more complete survey of complexity classes, in order of increasing rate of growth:

$O(1)$ Constant

The algorithm always takes the same amount of time, regardless of the size of the input -- very good, but rare. Occurs with algorithms that examine only a part of the input, e.g. algorithms that select a single element.

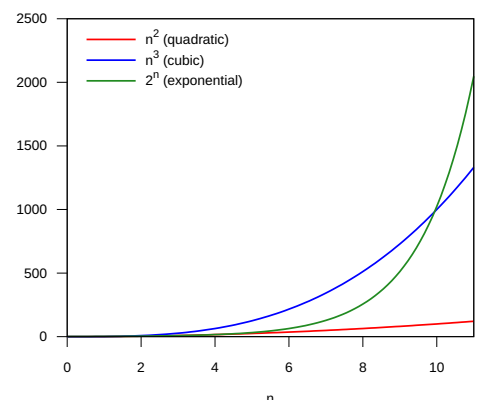
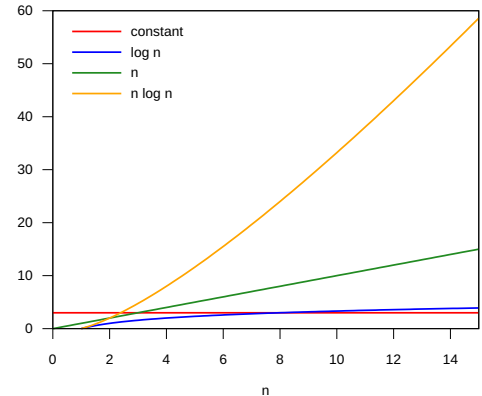
$O(\log n)$ Logarithmic

The amount of time taken grows logarithmically (good). This is typical of algorithms that repeatedly cut the size of the problem by some fraction, e.g. binary search.

$O(n)$ Linear

Time taken grows in direct proportion to input size (good/OK). Typically of algorithms making a limited number of passes over the input, e.g. sequential search, sum elements, find maximum. Any algorithm that examines all its input must take at least linear time.

$O(n \log n)$ Log-linear



e.g. fast sorting algorithms (week 7).

$O(n^2)$ Quadratic

The time taken grows in proportion to the square of the input size (OK/bad). This often occurs with algorithms involving two nested loops, e.g. AllDifferent (week 2), selection sort, insertion sort from last week.

$O(n^3)$ Cubic

The time taken grows in proportion to the cube of the input size (not so good). This often occurs with algorithms involving three nested loops, e.g. naive multiplication of n -by- n matrices.

$O(n^k)$ Polynomial (general case of the above)

This is the limit of what is usually considered "feasible", though it can be very expensive for larger k .

$O(2^n)$ Exponential

Time taken doubles each time the size increases by 1 (bad). This is typical of algorithms that examine all possible combinations from a set (exhaustive search), e.g. truth table, knapsack.

$O(n!)$ Factorial

$n! = 1 * 2 * 3 * \dots * n$ (even worse than exponential). This is typical of algorithms that examine all possible permutations from a set (exhaustive search), e.g. Hamiltonian path (travelling salesman).

When adding functions of different classes, we know that the lower ones in this list dominate the earlier ones, so the earlier ones can be ignored. For example, $n \log n + n^2$ is $O(n^2)$.
